BANKING MANAGEMENT SYSTEM

Project Report

Overview, implementation, and engineering assessment

CS-513 • System Software Mini Project

Source baseline: repository main branch + supplied source archive

Prepared 25 October 2026 | Version 1.0

https://github.com/Dev2907/Banking-Management-IIITB

PROJECT REPORT

Executive Summary

The Banking Management System is a terminal-based C++ application built around a TCP client/server model and a local SQLite database. The server is authoritative for authentication, role dispatch, banking state, loan processing, and feedback. It accepts connections on 127.0.0.1:8080 and creates a child process for each accepted client connection. Passwords are stored as libsodium password hashes, money is stored as integer smallest currency units, and account-changing operations use SQLite transactions.

Scope of this report

This document describes the current code and schema in the supplied repository/archive. It does not imply that every assignment item has been implemented or independently tested.

1. Project Objectives

- Provide role-based banking workflows for customers, employees, managers, and administrators.
- Demonstrate TCP socket programming and concurrent client handling on Linux.
- Persist users, accounts, transactions, loans, and feedback in SQLite.
- Use operating-system process and file-locking facilities to prevent duplicate simultaneous sessions for the same user.
- Preserve financial consistency with transactions and integer-based monetary values.

2. Technology Stack

Area	Current implementation
Language / standard	C++17
Platform APIs	Linux/POSIX sockets, fork(), flock(), open(), close(), waitpid()
Networking	IPv4 TCP; server binds to 127.0.0.1 port 8080
Persistence	SQLite file at src/db/main.db
Password security	libsodium password-hashing API (crypto_pwhash_str / verify)
Protocol	Newline-delimited text fields over a persistent TCP connection
Concurrency	Parent process accepts connections; child process handles each client

3. Implemented Role Capabilities

Role	Capabilities visible in current code
Customer	Sign up/login; select and create accounts; view balance; deposit; withdraw; transfer by recipient username; view up to 10 transaction records; apply for loan; submit feedback; change password; logout.
Employee	Change customer username; activate/deactivate customer login; view assigned loans; approve/reject an assigned loan; view users/accounts; view a customer transaction history; change password; logout.
Manager	Change customer username; activate/deactivate customer login; assign a pending loan to an active employee; view loans; view users/accounts; review customer feedback; change password; logout.
Administrator	Create customer, employee, or manager user; modify a user username, role, and active state; change password; logout.

4. System Architecture

The client presents menus and sends protocol fields. The server parent owns the listening socket and forks a child for each accepted connection. Each child opens a SQLite connection, authenticates the requested user, acquires a per-user flock session lock, and dispatches requests to the portal class for the authenticated role.

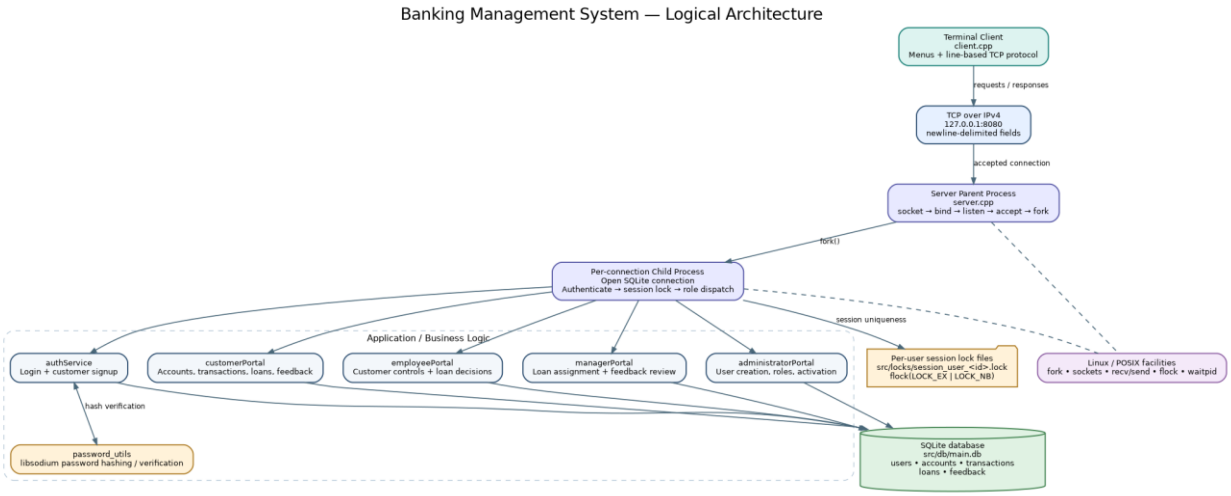


Figure 1. Logical architecture derived from the current source tree.

5. Data Model

The schema contains five tables. A user may own one or more accounts; an account has transaction records; a loan references the customer, account, assigned employee, and deciding user; feedback references its submitting user.

Table	Purpose / key relationships
users	Authentication identity, hashed password, role, active flag, creation timestamp.
accounts	Account owner (users.user_id), integer balance, creation timestamp.
transactions	Account ledger entries; integer amount, type, balance-after value, timestamp, optional paired transaction reference.
loans	Customer and account loan application, amount, status, assigned employee, decision maker and timestamps.
feedback	Customer-submitted message and timestamp.

6. Concurrency, Synchronization, and ACID

- The server uses fork() to run each client session in a child process.
- A per-user lock file is opened with mode 0600 and flock(LOCK_EX | LOCK_NB) prevents a second simultaneous session for the same user ID.
- Deposit, withdrawal, transfer, customer registration/account creation, loan application, and loan decision paths use SQLite transactions; write operations use BEGIN IMMEDIATE in the relevant implementations.
- Transfer is treated as a single unit: source debit, recipient credit, paired transaction entries, and reference linkage are rolled back together on failure.
- SQLite commit is the persistence boundary for the transaction paths. Durability should still be demonstrated with restart tests rather than assumed from source review alone.

Important distinction

The current source contains no explicit POSIX semaphore or pthread-based multithreading implementation. TCP sockets, fork(), flock(), and SQLite transactions are implemented. If the rubric requires every named OS mechanism individually, confirm that requirement with the instructor.

7. Security and Data Handling

- Password hashes are generated and verified with libsodium; plaintext passwords are not intentionally stored in users.password_hash.
- The SQL code predominantly uses prepared statements and bound parameters.
- The server retains the authenticated user ID/role in the child session and routes operations through server-side portal methods.
- Monetary values use integer smallest currency units rather than floating-point values. The CLI passes integer strings to stoll(), so users should enter whole smallest-unit amounts, not decimal strings.
- The TCP connection is not encrypted; the current server binds to loopback only. This is a coursework/demo design, not a production internet-banking deployment.

8. Main Gaps and Risks

Item	Current observation	Recommended follow-up
Foreign-key enforcement	Foreign keys are declared in schema, but the server connection setup does not enable PRAGMA foreign_keys=ON per connection.	Enable and test foreign key enforcement on every SQLite connection.
Response framing	Line-delimited protocol and list counts are used; authentication responses rely on message newlines for field separation.	Standardize a length/count-framed response format and test malformed/oversized inputs.

9. Conclusion

The current implementation demonstrates a clear role-portal separation, TCP client/server operation, per-connection processes, password hashing, file-lock session exclusivity, SQLite persistence, and transactional financial operations. The accompanying SRS, design description, diagrams, user/developer guide, and test plan provide a basis for explanation, review, and remaining validation before submission.